Лабораторная работа 16. Интеграция, release manifest и защита проекта

Версия: 2026–2027 учебный год Формат: команда 2–5 человек с обязательной индивидуальной защитой Продолжительность: 2 академических часа и самостоятельная подготовка Предшествующая работа: ЛР15 «Нагрузочный протокол, масштабирование и обзор LLMOps»

1. Цель

Собрать проверяемый итоговый выпуск сквозного ML-проекта, доказать связь кода, данных, модели, image, API и gates, выполнить end-to-end демонстрацию и индивидуально защитить инженерные решения.

2. Результаты обучения

Каждый студент должен уметь:

- прочитать и проверить release manifest;
- проследить commit → CI → image и data → run → model version;
- отличить alias `champion` от exact version;
- проверить SHA-256 evidence;
- объяснить status gate и управляемое исключение;
- выполнить диагностический шаг или rollback check;
- сформулировать ограничение и остаточный риск;
- доказать личный вклад в общей системе.

3. Командный результат

Команда предоставляет release tag `course-final-v1`:

1. работающий end-to-end репозиторий;
2. `reports/releases/lr16-manifest.json` со статусом `ready` или `ready_with_exception`;
3. успешный CI run и immutable image identity;
4. exact MLflow model version и проверенный alias `champion`;
5. актуальные model/data cards, security, monitoring и capacity evidence;
6. проверенный rollback и post-rollback smoke;
7. итоговый отчёт/README и сценарий демонстрации;
8. отдельная evidence card каждого участника.

4. Что не требуется повторять

Не нужно заново писать все pipeline. Используйте принятые артефакты ЛР1–ЛР15, исправляя только несогласованности. Финальная задача — связать identities и доказательства, а не создать параллельную копию проекта для защиты.

5. Freeze и preparation branch

Создайте issue финальной интеграции и короткоживущую branch. За 24 часа до защиты зафиксируйте content freeze: новые изменения проходят отдельный PR и повторяют затронутые gates. Не переписывайте историю после формирования manifest.

6. Инвентаризация identities

Зафиксируйте:

- full source commit SHA и release tag;
- data/feature contract versions и dataset identity;
- MLflow run ID, model name, version, alias и resolved URI;
- image digest либо full-commit tag;
- API/OpenAPI version;
- policy versions и environment.

Любое `latest`, «текущий» или «последний run» заменяется точной identity.

7. Копирование example spec

```
cp release/release_spec.example.json release/release_spec.json
```

Заполните реальными значениями команды. `release_spec.example.json` намеренно описывает draft и не может быть выдан за готовый выпуск.

8. Код и image

`source_commit` содержит 40 hex-символов. `image.ref` использует registry digest либо tag, равный этому SHA. SemVer можно добавить как удобный alias, но он не заменяет immutable identity.

9. Данные и признаки

Проверьте соответствие `data/contract.json`, DVC/источника, хронологического split и PIT audit. Manifest ссылается на evidence, а data/model cards объясняют происхождение, права и ограничения.

10. Модель

Разрешите alias `champion` и сохраните exact version. `resolved_uri` должен заканчиваться номером этой версии. Покажите MLflow run, tags `source_commit`, `data_sha256` и validation status.

11. API

Экспортируйте OpenAPI из release commit и выполните contract tests. Версия в schema должна совпасть с package/API version. Breaking change требует явной новой версии и плана миграции клиента.

12. Обязательные gates

Release policy требует:

1. data contract;
2. point-in-time features;
3. monitoring decision;
4. responsible AI;
5. capacity;
6. model card;
7. security.

Добавьте тесты/CI/API evidence по правилам команды, но не удаляйте обязательные gates.

13. Статусы gates

`pass` означает выполненную policy и доказательство. `accepted_with_investigation` допускается только с `owner`, `expires_at` и `reason`. `fail`, missing evidence или истёкшее исключение блокируют release.

14. Сборка manifest

```
python -m mlops_flight_delay.release build \
  --spec release/release_spec.json \
  --policy release/release_policy.json \
  --output reports/releases/lr16-manifest.json \
  --require-ready
```

Builder проверяет identities, наличие gates/evidence и вычисляет SHA-256 каждого файла.

15. Проверка manifest

```
python -m mlops_flight_delay.release verify \
  --manifest reports/releases/lr16-manifest.json \
  --policy release/release_policy.json
```

После изменения любого evidence manifest перестаёт проходить verify и должен быть пересобран с повторным review.

16. CI identity

Укажите URL успешного run для release commit. Локальный успешный запуск не заменяет командный CI. Если GitHub недоступен по независимой причине, преподаватель письменно принимает offline evidence; самостоятельно менять `status` на success запрещено.

17. Independent approval

`prepared_by` и `approved_by` различаются. Reviewer проверяет exact identities, hashes, исключения и rollback, а не только нажимает Approve. Автор может исправить замечание, но не подтверждает его сам.

18. Ready и ready_with_exception

Без CI и approval manifest остаётся `draft`. Все gates pass дают `ready`; принятое временное исключение — `ready_with_exception`. Команда объясняет влияние exception и план его закрытия.

19. Container/API smoke

Запустите release image и сохраните:

```
curl http://127.0.0.1:8080/healthz
curl http://127.0.0.1:8080/readyz
curl http://127.0.0.1:8080/metrics
python scripts/api_contract_smoke.py --base-url http://127.0.0.1:8080
```

Покажите valid/invalid request, request ID, resolved model identity и отсутствие raw features в log.

20. Post-release checks

Проверьте readiness, error rate, latency и model identity после запуска. Deployment command без фактической проверки сервиса не считается успешным release.

21. Rollback preparation

До drill зафиксируйте предыдущие exact model/image identities, причину/trigger, approver и smoke criteria. Если совместимая предыдущая пара отсутствует, release не готов к защите.

22. Rollback drill

Переместите alias `champion` на проверенную предыдущую model version и верните предыдущий immutable image либо выполните согласованный локальный эквивалент. Не используйте production и не удаляйте данные. Сохраните from/to identities, timestamps и результат.

23. Проверка после rollback

Выполните smoke prediction, `/readyz`, contract check и проверку resolved model. Один успешный deployment status не доказывает работоспособность. После drill верните согласованное состояние новым управляемым действием и зафиксируйте его.

24. Artifact attestation — расширение

Если репозиторий/тариф поддерживает attestations, сформируйте provenance для release artifact и проверьте `gh attestation verify`. Это дополнительный балл evidence, но не замена tests, security, model quality и release approval.

25. Итоговый README/отчёт

Он содержит задачу, архитектуру, quick start, contracts, pipeline, monitoring, ограничения, responsible use, release/rollback, ссылки на manifest и распределение вкладов. Приватный СКИФ не используется вместо самодостаточного описания.

26. Сценарий командной демонстрации

Используйте `docs/final-demo-script.md`. Продолжительность 10–12 минут. Назначьте ведущего каждого шага и запасной вариант на случай сетевой недоступности. Запись экрана не заменяет live explanation.

27. Роли команды

Для 2–5 человек:

- release/data/model identity owner;
- API/container/integration owner;
- CI/evidence/manifest owner;
- observability/governance/capacity owner;
- rollback/security/demo reviewer.

В паре роли совмещаются. Каждый участник выполняет технический или проверочный шаг; роль только ведущего презентации не является индивидуальным вкладом.

28. Индивидуальная сдача участника команды

Командный результат: `course-final-v1`, готовый manifest, CI, demo и rollback evidence.

Каждый заполняет `docs/evidence/lr16-evidence-card.md`; совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Release	tag, manifest SHA и source commit
Роль	роль и отличие от ЛР15
Задача	собственная issue с критерием приёмки
Личный артефакт	integration fix, gate, evidence, demo step, rollback или review

Поле	Что указать
Вклад	конкретный diff/решение и его связь с release
Команда	индивидуальный воспроизводимый запуск и output
Решение	альтернатива, критерий, риск и ограничение
Диагностика	действие преподавателя и результат
Review	замечание другого участника и реакция команды
Использование ИИ	инструмент и самостоятельно проверенные утверждения

29. Минимальный допуск

1. Release tag и full commit SHA зафиксированы.
2. Manifest имеет `ready` или `ready_with_exception`.
3. CI run успешен либо есть письменное решение преподавателя.
4. У студента собственная issue и встроенный личный артефакт.
5. PR прошёл review другого участника.
6. Evidence card содержит рабочие identities/ссылки.
7. Rollback подготовлен и безопасен.
8. Студент способен выполнить диагностическую операцию.

30. Индивидуальная защита

За 3–4 минуты студент:

1. показывает личный артефакт в release tag;
2. прослеживает его до manifest/gate;
3. выполняет одну операцию: verify hash, resolve alias, contract smoke, broken-copy gate, readiness check, интерпретация alert или post-rollback smoke;
4. объясняет решение, альтернативу и остаточный риск;
5. отвечает на вопрос по соседнему связанному компоненту.

31. Набор диагностических заданий преподавателя

- заменить image ref на `latest` в копии spec и объяснить отказ;
- изменить evidence и запустить verify;
- показать exact version за alias;
- определить, почему manifest остаётся draft;
- найти owner/expiry исключения;
- сопоставить API version с image;
- выбрать rollback или roll-forward;
- найти поле, которое нельзя логировать;
- объяснить, почему attestation не доказывает качество модели.

32. Стоп-факторы

- release основан на `latest` или неизвестном commit;

- alias выдан без exact model version;
- manifest вручную объявлен ready без CI/approval;
- pass не имеет evidence/hash;
- исключение бессрочно или без владельца;
- author и approver совпадают;
- rollback не содержит точных identities;
- секреты/персональные данные опубликованы;
- общая демонстрация выдана за личную защиту;
- участник не объясняет собственный артефакт.

33. Повторная сдача

Переоткройте issue, исправьте отдельным PR, повторите затронутые gates, пересоберите manifest и обновите evidence card. Создайте новый release tag; старый не перемещается и первая попытка не удаляется. Повторно защищается исправленная и связанная с ней часть.

34. Критерии оценивания

Критерий	Баллы	Полное выполнение
Release identities и manifest	2	code/data/model/image/API связаны точно
Gates, CI и approvals	2	evidence проверяемо, исключения управляются
End-to-end демонстрация	2	запуск, API, monitoring и ограничения показаны
Rollback и post-check	2	exact previous state восстановлено и проверено
Индивидуальный вклад и защита	2	личный артефакт, review и диагностика доказаны

Максимум: 10 баллов. Порог зачёта — 6 баллов при отсутствии стоп-факторов. Командная и индивидуальная оценки могут различаться.

35. Чек-лист

- [ ] Команда 2–5 человек, у каждого техническая/проверочная роль.
- [ ] Release tag не перемещается.
- [ ] Full source commit записан.
- [ ] Data/feature identities согласованы.
- [ ] MLflow run, exact version и alias разрешены.
- [ ] Image immutable и соответствует commit.
- [ ] API/OpenAPI version совпадает.
- [ ] Обязательные gates присутствуют.
- [ ] Evidence files существуют и hashes проверяются.
- [ ] Исключения имеют owner/expiry/reason.
- [ ] CI успешен и approval независим.

- [ ] Manifest не draft.
- [ ] Container/API smoke выполнен.
- [ ] Rollback и post-check воспроизводимы.
- [ ] Cards каждого участника уникальны.
- [ ] Итог отмечен `course-final-v1`.

36. После защиты

Команда сохраняет manifest, отчёт, cards и ссылки на CI согласно правилам курса. Секреты и реальные данные не архивируются. Незакрытые исключения передаются владельцу; учебный сервис останавливается, если его дальнейшая работа не согласована.

37. Официальные источники

- GitHub artifact attestations: https://docs.github.com/en/actions/concepts/security/artifact-attestations
- GitHub deployments/environments: https://docs.github.com/en/actions/how-tos/deploy/configure-and-manage-deployments
- GitHub Actions security: https://docs.github.com/en/actions/how-tos/secure-your-work
- MLflow Model Registry workflow: https://mlflow.org/docs/latest/ml/model-registry/workflow/